MINICURSO PRÁTICO • DESENVOLVIMENTO SEGURO & HACKING ÉTICO

Segurança em Aplicações Web: SQLi, IDOR, XSS & CTF

Análise de vulnerabilidades em Python/Flask, técnicas de remediação e desafio prático de exploração de flags.

>_ Ambiente: Kali Linux + Flask + SQLite

15º tecvale
ENCONTRO TECNOLÓGICO DO VALE DO SÃO FRANCISCO

TECNOLOGIA PARA O DESENVOLVIMENTO ECONÔMICO E SOCIAL

26 A 28 OUT
UNIRIOS



Prof. Erick B. Nascimento

erick.nascimento@unirios.edu.br

SISTEMAS DE INFORMAÇÃO

UniRios

PROGRAMA DO MINICURSO

Módulos de Estudo e Prática

1. SQL Injection

Bypass de login e extração indevida de dados no banco.

2. IDOR

Acesso direto e não autorizado a recursos de terceiros.

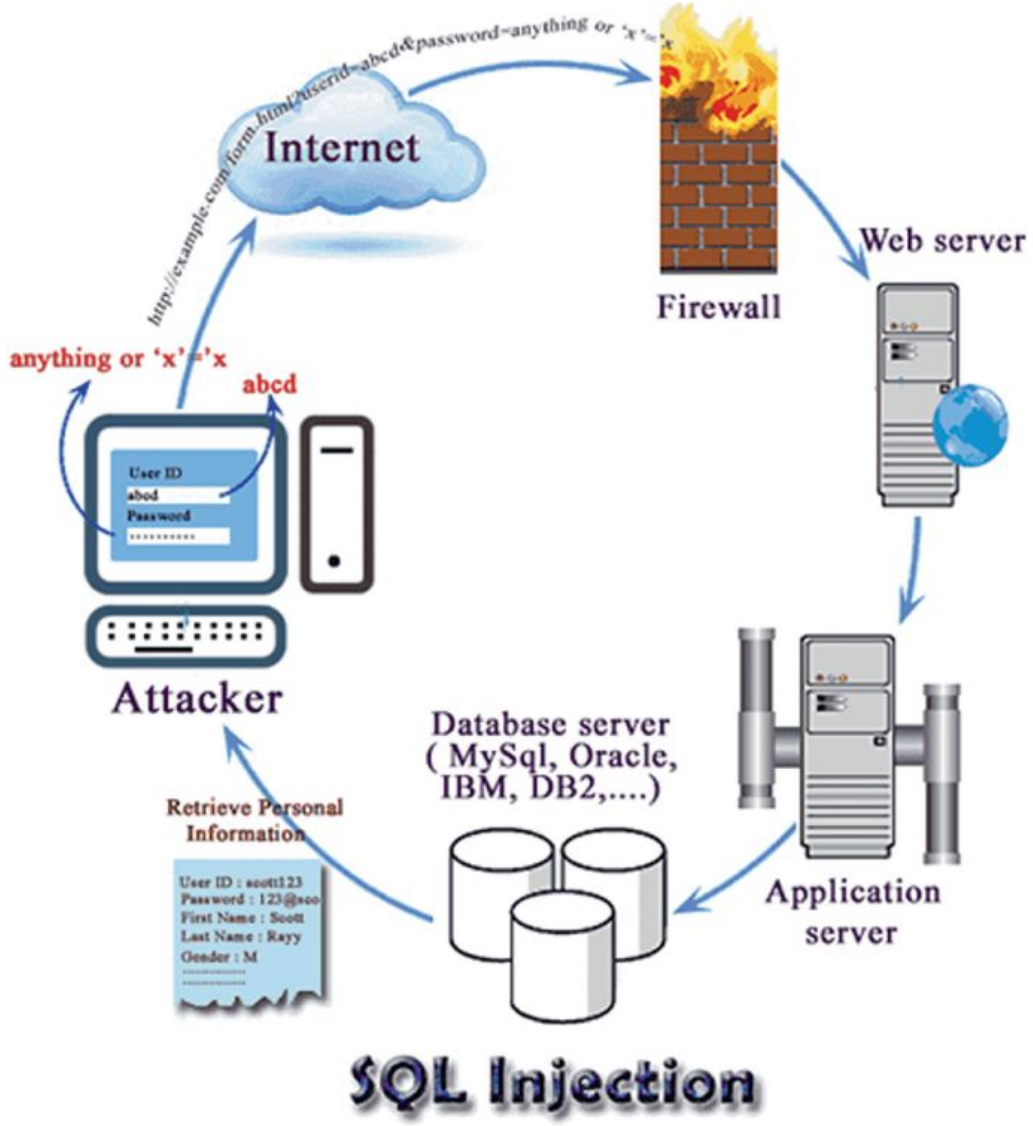
3. XSS (Jinja & Bleach)

Injeção de scripts e sanitização de HTML no Flask.

4. Desafio CTF

Atividade prática para capturar flags de validação.

O que é SQL Injection (SQLi)?



Confusão de Contexto
O servidor interpreta dados de entrada como instruções SQL.

Manipulação de Sintaxe
Caracteres como aspas simples alteram a lógica da consulta.

```
SELECT USER NAME, EMAIL,TEL
FROM USERS
WHERE USER NAME = 'Ken Sato'
AND PASSWORD = '' OR 'a' = 'a' ; --;
```

Impacto Principal
Bypass de autenticação e leitura não autorizada de dados.

MÓDULO 1 • CÓDIGO EM PYTHON

SQLi: Interpolação Insegura vs. Consulta Parametrizada

❌ Código Vulnerável (Concatenação)

```
# Entrada do usuário inserida diretamente

query = f"SELECT * FROM users WHERE email = '{email}' AND
password = '{password}'"

user = db.session.execute(text(query)).fetchone()
```

Risco: Payload `' OR '1'='1` altera a consulta.

✅ Código Seguro (Prepared Statement)

```
# Uso de marcadores de posição

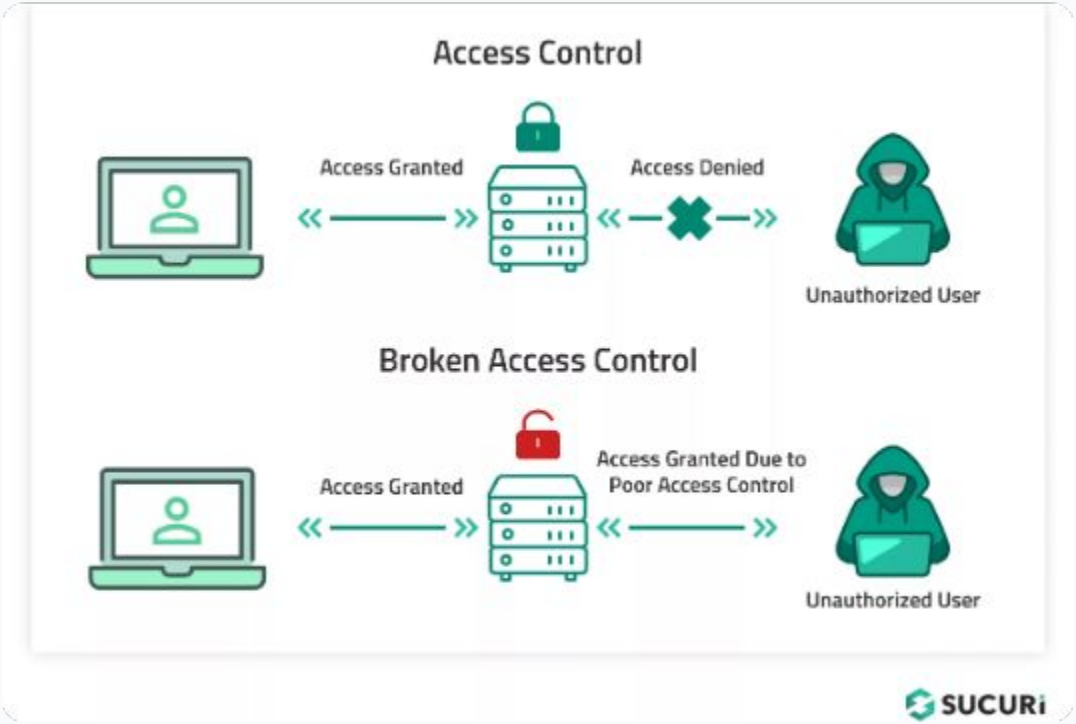
query = text("SELECT * FROM users WHERE email = :email AND
password = :password")

user = db.session.execute(query, {"email": email,
"password": password}).fetchone()
```

Solução: O banco trata entradas puramente como texto.

MÓDULO 2 • CONTROLE DE ACESSO

O que é IDOR (Insecure Direct Object Reference)?



Referências Expostas
Identificadores internos (IDs) expostos em URLs ou APIs.

Ausência de Autorização
O sistema não valida se o usuário autenticado é o dono do recurso.

Exemplo Clássico
Alterar `/fatura/101` para `/fatura/102` e visualizar dados de terceiros.

MÓDULO 2 • CÓDIGO EM PYTHON

IDOR: Falha de Checagem vs. Validação de Posse

❌ Rota Vulnerável a IDOR

```
@app.route('/fatura/')
def ver_fatura(id):
  doc = Fatura.query.get(id)
  return render_template('fatura.html', doc=doc)
```

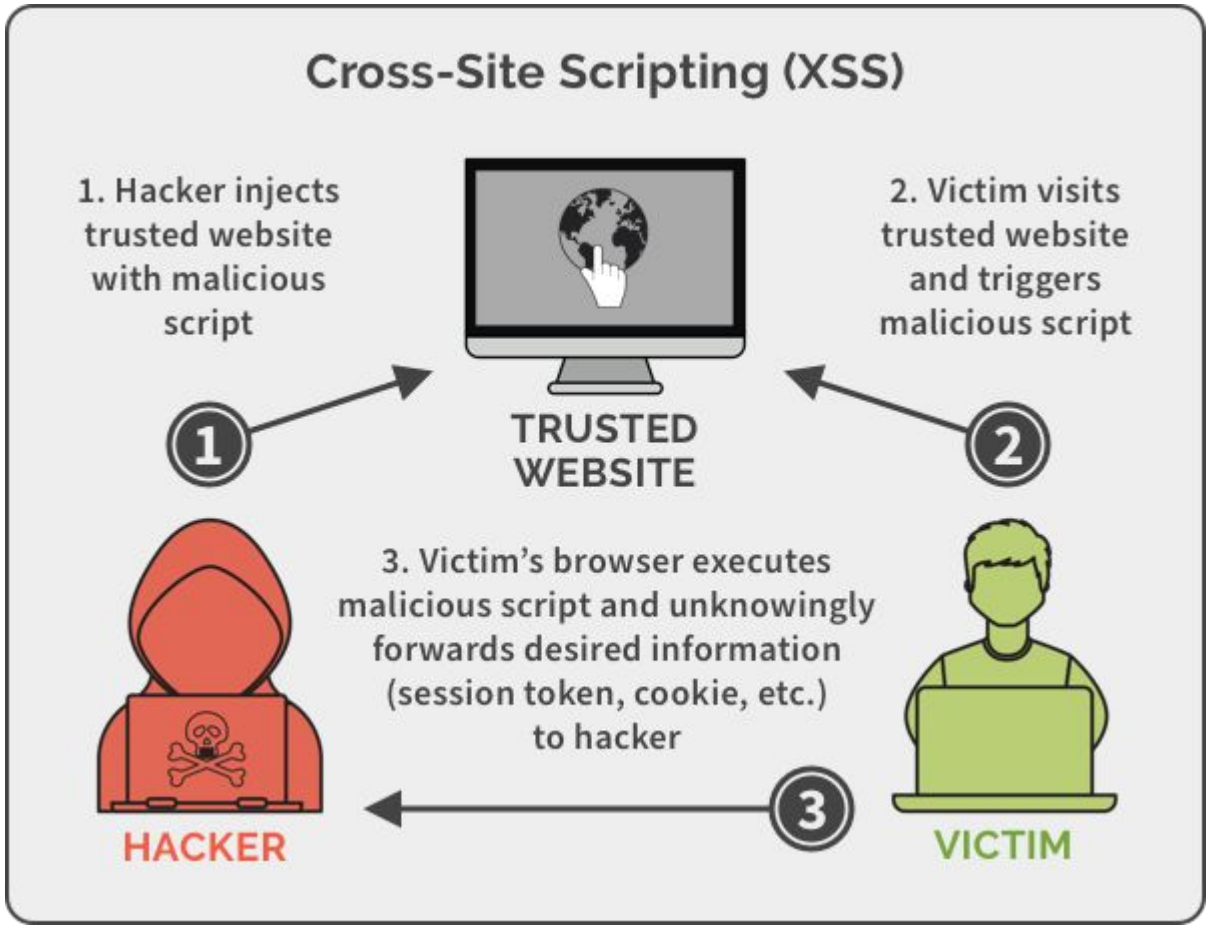
Falha: Qualquer usuário acessa qualquer ID de fatura.

✅ Rota Protegida (Validação de Sessão)

```
@app.route('/fatura/')
def ver_fatura(id):
  doc = Fatura.query.filter_by(id=id,
usuario_id=session['user_id']).first_or_404()
  return render_template('fatura.html', doc=doc)
```

Solução: Restringe a busca estritamente ao dono do recurso.

O que é Cross-Site Scripting (XSS)?



Injeção no Navegador
Inserção de scripts maliciosos (JavaScript) nas páginas exibidas.

Execução no Cliente
O código roda no navegador da vítima, no contexto da sessão ativa.

Risco de Roubo de Sessão
Permite roubar cookies de autenticação e redirecionar usuários.

MÓDULO 3 • PROTEÇÃO CONTRA XSS

Defesa em Python: Autoescaping Jinja2 e Biblioteca Bleach

1. Escaping Padrão do Jinja2

Por padrão, o Jinja2 converte caracteres perigosos em entidades HTML HTML safe (`<` vira `<`).

```
{{ comentario }}

{{ comentario | safe }}
```

2. Sanitização HTML com Bleach

Permite aceitar tags HTML seguras (ex: ``) removendo scripts maliciosos.

```
import bleach
clean_html = bleach.clean(entrada,
  tags=['b', 'i', 'u'],
  strip=True)
```

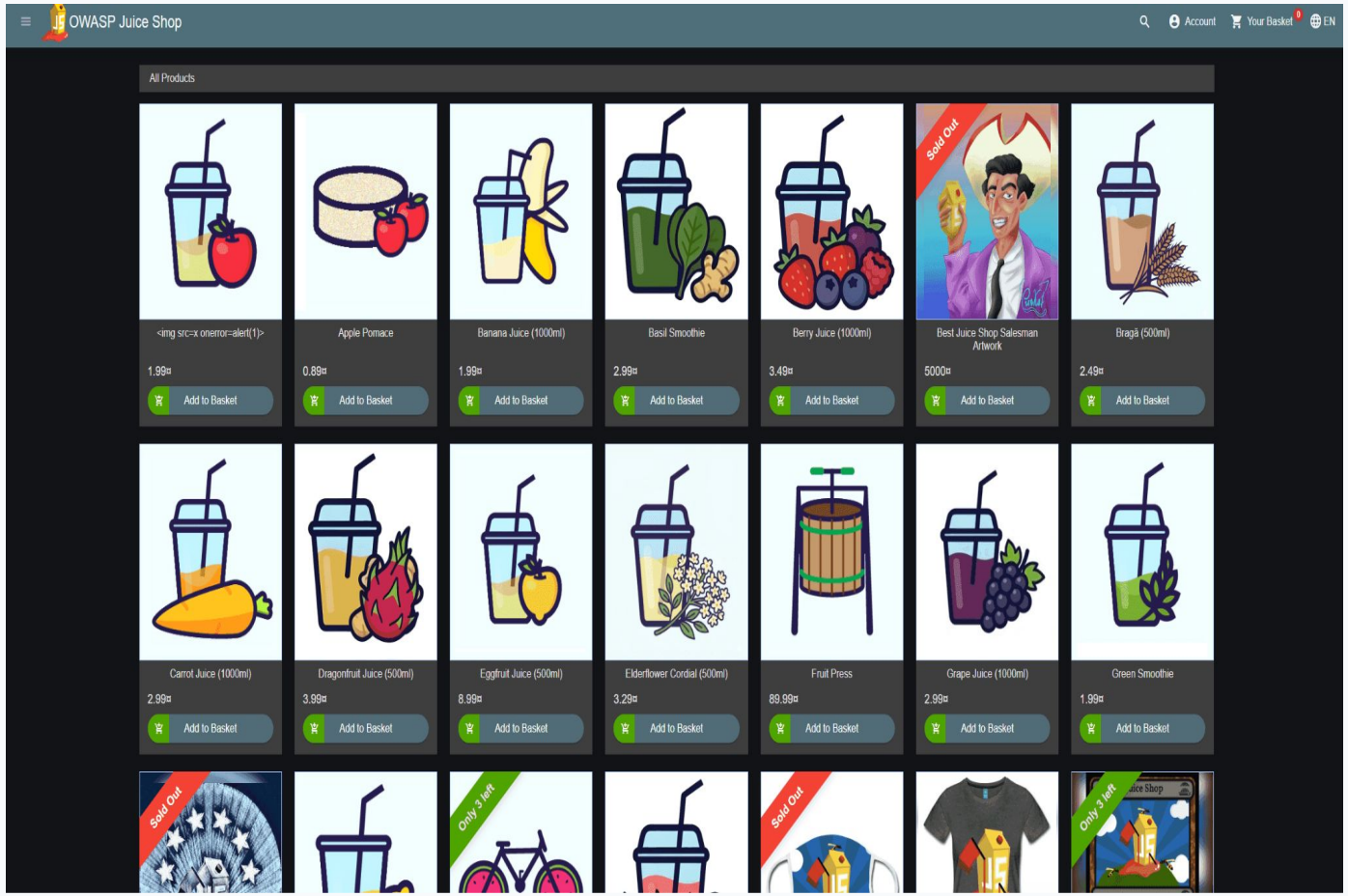
RESUMO TEÓRICO

Comparativo de Vulnerabilidades Web

Vulnerabilidade	Alvo Principal	Causa Raiz	Mecanismo de Defesa
SQL Injection	Banco de Dados (SGBD)	Concatenação de SQL bruto	Consultas Parametrizadas
IDOR	Controle de Acesso	Ausência de checagem de dono	Validação de sessão do usuário
XSS	Navegador do Cliente	Injeção de scripts HTML/JS	Jinja Escaping & Bleach Clean

FERRAMENTA DE PRÁTICA VIA INTERFACE GRÁFICA

OWASP Juice Shop



O **OWASP Juice Shop** é uma aplicação web intencionalmente vulnerável desenvolvida pela OWASP (Open Web Application Security Project). Ela simula um e-commerce moderno construído em Node.js, Express e Angular, recheado de falhas de segurança do mundo real.

- **Objetivo Didático:** Em vez de explicar conceitos apenas na teoria, o Juice Shop permite que os alunos explorem e explorem vulnerabilidades na prática, em um ambiente controlado e legal.
- **Sistema de CTF (Capture The Flag):** A aplicação possui um *Score Board* escondido que detecta automaticamente quando uma vulnerabilidade é explorada com sucesso, liberando conquistas que vão do nível iniciante (nível 1) ao avançado (nível 6).
- **Abrangência:** Cobre todas as vulnerabilidades do **OWASP Top 10** (como Injeção, Falhas de Autenticação, Controle de Acesso Quebrado e XSS).

MÓDULO 4 • ATIVIDADE PRÁTICA

Como Funciona a Capture The Flag (CTF)?



1. Exploração Handson das rotas vulneráveis no Kali Linux
2. Localização de segredos ocultos (Flags) no sistema
3. Submissão das chaves no painel de pontuação da turma
4. Aplicações de patches no código Python para bloquear falhas

MÓDULO 4 • REGRAS DO DESAFIO

Formato das Flags e Pontuação

FLAG 01 • SQLi

Bypass de Login

Autenticar como administrador via payload SQL.

`FLAG{sqli_bypass_admin}`

FLAG 02 • IDOR

Acesso a Documento

Acessar o arquivo confidencial alterando o ID da URL.

`FLAG{idor_private_doc}`

FLAG 03 • XSS

Injeção de Script

Executar Script e capturar o cookie de sessão simulado.

`FLAG{xss_cookie_stolen}`

MÃOS À OBRA

Início da Etapa Prática & CTF!

Abram seus terminais no Kali Linux, ativem o ambiente virtual e iniciem a aplicação do laboratório.

🚩 **Boa sorte na busca pelas Flags!**